E-Commerce Database System

Final Project Plan

Stack: Django + MySQL (Cloud) + Vercel

1. Project Overview

College DBMS project: Online shopping backend with Product Catalog, Cart System, Order Placement, and Payment Tracking. Full-stack web app using Django (Python), MySQL database connected via Django ORM and raw SQL, deployed on Vercel with cloud-hosted MySQL for production.

2. Tech Stack

Layer	Technology	Purpose
Backend	Django 5.x	MVT framework, ORM, Admin, Auth
Database	MySQL 8 (Cloud)	Relational DB - joins, triggers, views
DB Driver	mysqlclient	Python-MySQL connection
Frontend	Django Templates + Bootstrap 5	Server-rendered UI
Auth	Django built-in User model	Login, sessions, permissions
Images	Cloudinary	Product images (Vercel has no local disk)
Config	python-decouple / .env	Secrets not in Git
Deploy App	Vercel	Serverless Django + CDN static files
Deploy DB	TiDB Serverless / Railway	Free-tier cloud MySQL

3. Why This Stack Works

- Vercel officially supports Django (zero-config, detects manage.py).
- Same Django code runs locally and on Vercel - only DB_HOST changes.
- Local MySQL for dev; Cloud MySQL for production (laptop DB not reachable from Vercel).
- Django ORM for CRUD plus raw SQL files for viva (views, triggers, procedures).
- Django Admin gives free product/order management for demo.

4. Django Project Structure

```
ecommerce_dbms/
|-- manage.py
|-- requirements.txt
|-- .env.example          (template - never commit .env)
|-- pyproject.toml          (optional Vercel config)
|-- config/                 settings, urls, wsgi
|-- accounts/               User profile, register, login
|-- catalog/                Category, Product models + views
|-- cart/                   Cart, CartItem
|-- orders/                 Order, OrderItem, Payment
|-- sql_scripts/            schema.sql, views.sql, triggers.sql
|-- templates/              base.html, product list, cart, checkout
|-- static/                 CSS, Bootstrap
```

5. Database Tables (Detailed)

Table	Key Columns	Notes
user (auth_user)	id, username, email	Django built-in + Profile extend
category	id, name	Electronics, Clothing, etc.
product	id, category_id, price, stock	INDEX on category_id
cart	id, user_id	One cart per logged-in user
cart_item	cart_id, product_id, qty	UNIQUE(cart_id, product_id)
order	id, user_id, total, status	status: pending/confirmed/shipped/delivered
order_item	order_id, product_id, unit_price	Price snapshot at order time
payment	order_id, amount, status	status: pending/success/failed/refunded
address	user_id, line1, city, pincode	Shipping address

Design 1: Database ER Schema

Relational schema for Product Catalog, Cart, Orders, and Payment Tracking. All tables use InnoDB engine with foreign keys and indexes for viva demonstration.

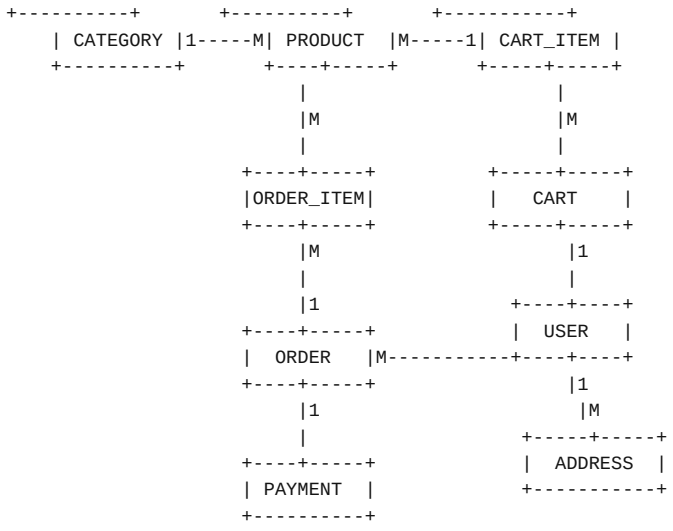
Entity	Attributes
USER	id PK, username UK, email UK, password_hash, phone, created_at
ADDRESS	id PK, user_id FK->USER, line1, city, state, pincode, is_default
CATEGORY	id PK, name UK, description
PRODUCT	id PK, category_id FK, name, description, price, stock_qty, image_url, is_active
CART	id PK, user_id FK->USER, updated_at
CART_ITEM	id PK, cart_id FK, product_id FK, quantity (UNIQUE cart+product)
ORDER	id PK, user_id FK, address_id FK, total_amount, status, order_date
ORDER_ITEM	id PK, order_id FK, product_id FK, quantity, unit_price, subtotal
PAYMENT	id PK, order_id FK UK, method, amount, status, transaction_id, paid_at

Relationships

- USER 1---M CART, ORDER, ADDRESS

- CATEGORY 1---M PRODUCT
- CART 1---M CART_ITEM ---M 1 PRODUCT
- ORDER 1---M ORDER_ITEM ---M 1 PRODUCT
- ORDER 1---1 PAYMENT
- ORDER_ITEM.unit_price = price snapshot at order time

ER Diagram (Visual)



Design 2: System Architecture

Production (Vercel + Cloud MySQL)

```
[ Browser / Mobile ]
            |
            v HTTPS
    +--------------------------+
    |      VERCEL PLATFORM     |
    |  CDN (Static CSS/JS/IMG) |
    |  Django Vercel Function  |
    |  (WSGI - Gunicorn-like)  |
    +-------------+------------+
                  |
                  | mysqlclient / PyMySQL
                  | SSL connection
                  v
    +--------------------------+
    |   CLOUD MYSQL (TiDB /    |
    |   Railway / Aiven)       |
    |   Tables + Views +       |
    |   Triggers + Procedures  |
    +--------------------------+

    External: Cloudinary (product images)
    Env Vars: DB_HOST, DB_USER, DB_PASS, DB_NAME, SECRET_KEY
```

Local Development

```
[ Browser ] --> python manage.py runserver --> MySQL localhost:3306
    Same Django code | Different DB_HOST in .env
```

Order Placement Flow (Transaction)

- 1. User clicks Checkout -> POST /orders/place/
- 2. Django @transaction.atomic block starts
- 3. Validate cart items and stock quantities
- 4. INSERT into ORDER and ORDER_ITEM tables
- 5. UPDATE PRODUCT stock (or trigger fires)
- 6. INSERT PAYMENT record (status = pending)
- 7. Clear CART_ITEM rows
- 8. COMMIT - all or nothing (ROLLBACK on error)

6. Feature Implementation Map

Feature	Django App	Key Models / Views	SQL Concept
Product Catalog	catalog	Category, Product, ListView	SELECT + JOIN
Cart System	cart	Cart, CartItem, add/remove	INSERT/UPDATE/DELETE
Order Placement	orders	place_order view	TRANSACTION
Payment Tracking	orders	Payment model	FK + status enum
Admin Panel	Django Admin	All models registered	CRUD demo
Reports (Viva)	sql_scripts	Raw SQL views	VIEW, GROUP BY

7. Environment Variables

```
# Local (.env)
SECRET_KEY=your-secret-key
DEBUG=True
DB_ENGINE=django.db.backends.mysql
DB_NAME=ecommerce_db
DB_USER=root
DB_PASSWORD=yourpassword
DB_HOST=127.0.0.1
DB_PORT=3306
CLOUDINARY_URL=cloudinary://...

# Vercel Dashboard (Production)
SECRET_KEY=<strong-random-key>
DEBUG=False
DB_HOST=<cloud-mysql-host>
DB_USER=<cloud-user>
DB_PASSWORD=<cloud-password>
DB_NAME=ecommerce_db
DB_PORT=3306
ALLOWED_HOSTS=.vercel.app
```

8. Phase-wise Timeline (6 Weeks)

Week	Tasks	Deliverable
1	Django setup, MySQL connect, User auth	Login/Register working
2	Category + Product CRUD, Admin seed data	Product catalog live
3	Cart add/remove/update quantity	Cart system complete

4	Checkout, Order + Payment, transactions	Order placement working
5	SQL views, triggers, indexes, report page	DBMS viva material
6	Cloud MySQL + Vercel deploy, testing, report	Live demo URL

9. Deployment Steps

- Step 1: Push code to GitHub (.env in .gitignore).
- Step 2: Create TiDB Serverless or Railway MySQL instance.
- Step 3: Run sql_scripts/schema.sql on cloud DB.
- Step 4: Import repo on vercel.com - auto-detects Django.
- Step 5: Add all env vars in Vercel Project Settings.
- Step 6: Deploy -> run migrations via Vercel CLI or local against cloud DB.
- Step 7: Create superuser, load seed products, test live URL.

10. Vercel Limitations (Know Before Demo)

- Cold start: first request may take 1-2 seconds.
- No local MEDIA_ROOT - use Cloudinary for uploads.
- No Celery/background jobs on Vercel.
- Function timeout: ~10s (Hobby) / ~60s (Pro).
- SQLite must NOT be used in production on Vercel.

11. Viva / Report Checklist

- ER Diagram + Architecture Diagram (this PDF + designs.html).
- CREATE TABLE script with PK, FK, constraints.
- At least 1 VIEW (e.g. sales by category).
- At least 1 TRIGGER (e.g. reduce stock on order).
- TRANSACTION demo during order placement.
- EXPLAIN on indexed vs non-indexed query.
- Screenshots: Admin, Catalog, Cart, Order, Payment status.
- Live Vercel URL in report.

12. Team Role Suggestion (Optional)

Member	Responsibility
Member 1	Database schema, SQL scripts, ER diagram

Member 2	Django models, migrations, Admin
Member 3	Views, Templates, Bootstrap UI
Member 4	Cart + Order logic, transactions, deployment

Also open docs/designs.html in browser and use Ctrl+P -> Save as PDF for high-quality visual diagrams.